



南開大學
Nankai University

计算机学院
数据安全第二次实验报告

姓名：周重天
学号：2311082
专业：计算机科学与技术

5th May 2026

目录

1 实验要求	2
2 实验原理	2
3 实验过程	2
3.1 libsnark 环境搭建	2
3.2 算术电路设计与拍平	7
3.3 电路核心代码实现 (common.hpp)	7
3.4 密钥生成程序 (mysetup.cpp)	8
3.5 证明生成程序 (myprove.cpp)	8
3.6 验证程序 (myverify.cpp)	9
3.7 编译运行与配置	9
4 总结	11

1 实验要求

参考教材实验 3.1 流程，利用 libsnark 开发框架完成零知识证明方案的实现。具体目标为：证明者 Alice 拥有一个私密整数 $x = 3$ ，她希望在不泄露 x 具体数值的前提下，向验证者证明其掌握方程 $x^3 + x + 5 = 35$ 的解。

2 实验原理

zkSNARK (zero-knowledge Succinct Non-interactive Arguments of Knowledge) 是一类基于 CRS 模型实现的典型非交互式零知识证明技术。其技术特征包括：简洁性、无交互、可靠性以及零知识性。

实现 zkSNARK 应用的开发步骤大体如下：定义电路（将内容用算术电路表示）、将电路表达为 RICS (一阶约束系统)、生成密钥 (PK 和 VK)、生成证明（使用证明密钥构造证明）以及验证证明（使用验证密钥校验）。

3 实验过程

3.1 libsnark 环境搭建

libsnark 是用于开发 zkSNARK 应用的 C++ 代码库。其安装涉及多个子模块，具体步骤复刻如下：

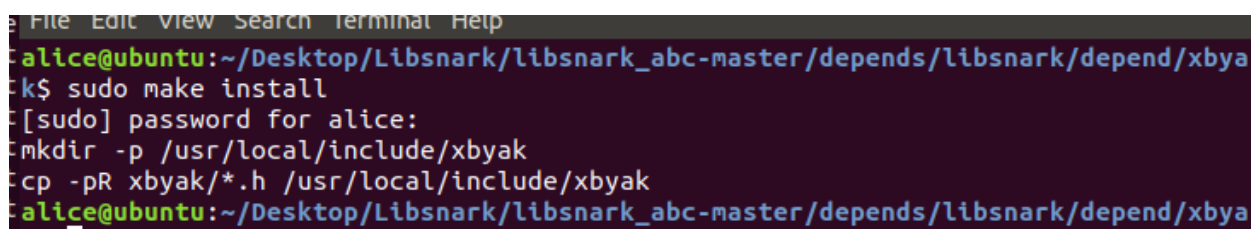
- 1) 创建名为 Libsnark 的文件夹。
- 2) 下载 libsnark_abc 并解压到 /Libsnark/libsnark_abc-master。
- 3) 下载 libsnark 源码并将文件复制到 depends/libsnark 文件夹内。
- 4) 下载并解压六个子模块 (Libffft、Libff、Gtest、Xbyak、Ate-pairing、Libsnark-supercop)。
- 5) 安装依赖项：

```
1 sudo apt install build-essential cmake git libgmp3-dev libprocps-dev python3-markdown
libboost-program-options-dev libssl-dev python3 pkg-config
```

6) 安装子模块 xbyak: 将下载得到的文件夹 Xbyak 内的文件复制到 /Libsnark/libsnark_abc-master/depends/libsnark 并在该目录下打开终端，执行以下命令

```
1 # 在 depends/libsnark/depends/xbyak 目录下
2 sudo make install
```

出现以下代码，说明安装成功。



```
File Edit View Search Terminal Help
alice@ubuntu:~/Desktop/Libsnark/libsnark_abc-master/depends/libsnark/depend/xbyak
k$ sudo make install
[sudo] password for alice:
mkdir -p /usr/local/include/xbyak
cp -pR xbyak/*.h /usr/local/include/xbyak
alice@ubuntu:~/Desktop/Libsnark/libsnark_abc-master/depends/libsnark/depend/xbyak
```

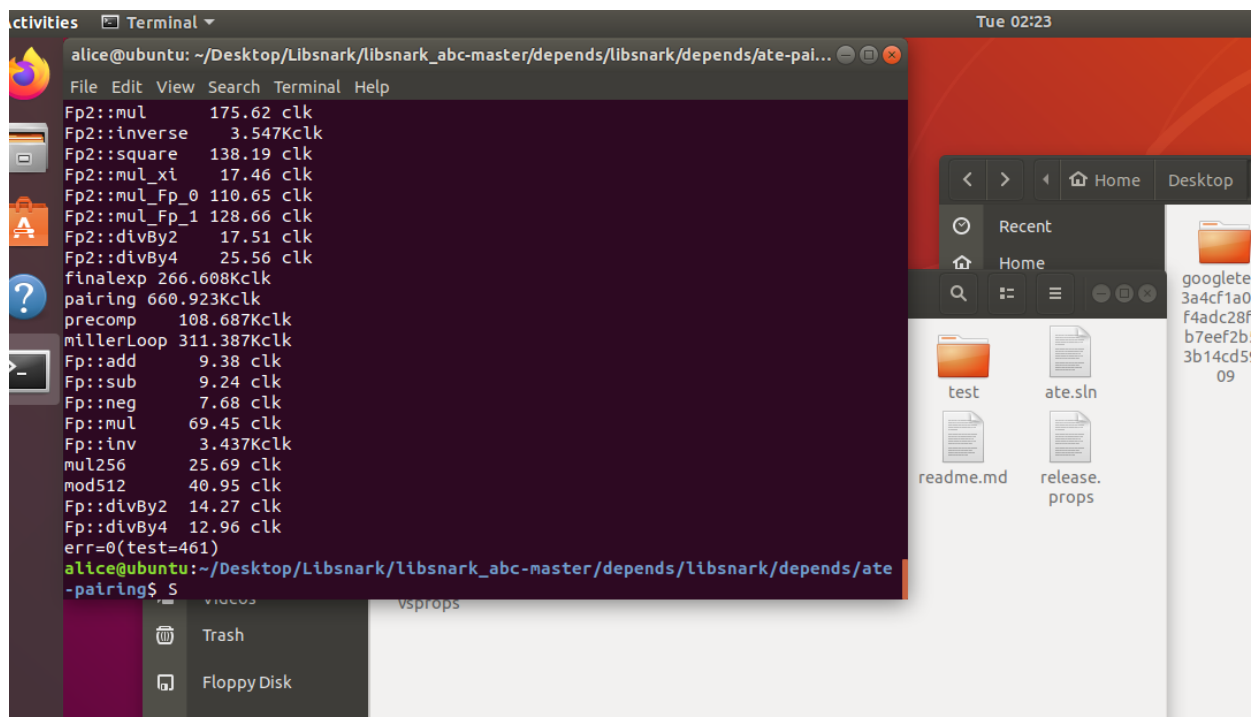
- 7) 安装子模块 ate-pairing:

```
1 # 在 depends/libsnark/depends/ate-pairing 目录下
2 make -j
3 test/bn
```

执行 make -j 最后几行结果如下：

```
g++ -o loop_test loop_test.o -lm -lzm -L../lib -lgmp -lgmpxx -m64
g++ -o java_api java_api.o -lm -lzm -L../lib -lgmp -lgmpxx -m64
g++ -o sample sample.o -lm -lzm -L../lib -lgmp -lgmpxx -m64
g++ -o test_zm test_zm.o -lm -lzm -L../lib -lgmp -lgmpxx -m64
g++ -o bench_test bench.o -lm -lzm -L../lib -lgmp -lgmpxx -m64
g++ -o bn bn.o -lm -lzm -L../lib -lgmp -lgmpxx -m64
make[1]: Leaving directory '/home/alice/Desktop/Libsnark/libsnark_abc-master/depends/libsnark/depends/ate-pairing/test'
alice@ubuntu:~/Desktop/Libsnark/libsnark_abc-master/depends/libsnark/depends/ate-pairing$
```

执行 test/bn 最后几行结果如下：

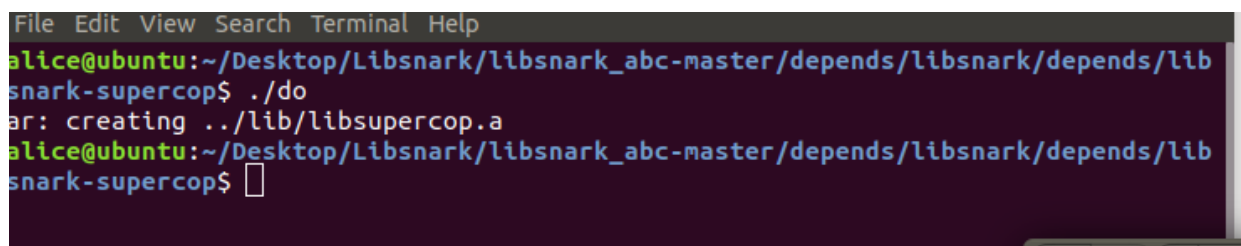


```
activities Terminal Tue 02:23
alice@ubuntu: ~/Desktop/Libsnark/libsnark_abc-master/depends/libsnark/depends/ate-pai...
File Edit View Search Terminal Help
Fp2::mul 175.62 clk
Fp2::inverse 3.547Kclk
Fp2::square 138.19 clk
Fp2::mul_xi 17.46 clk
Fp2::mul_Fp_0 110.65 clk
Fp2::mul_Fp_1 128.66 clk
Fp2::divBy2 17.51 clk
Fp2::divBy4 25.56 clk
finalexp 266.608Kclk
pairing 660.923Kclk
precomp 108.687Kclk
millerLoop 311.387Kclk
Fp::add 9.38 clk
Fp::sub 9.24 clk
Fp::neg 7.68 clk
Fp::mul 69.45 clk
Fp::inv 3.437Kclk
mul256 25.69 clk
mod512 40.95 clk
Fp::divBy2 14.27 clk
Fp::divBy4 12.96 clk
err=0(test=461)
alice@ubuntu:~/Desktop/Libsnark/libsnark_abc-master/depends/libsnark/depends/ate-pairing$
```

8) 安装子模块 libsnark-supercop:

```
1 # 在 depends/libsnark/depends/libsnark-supercop 目录下
2 ./do
```

出现以下代码，说明安装成功



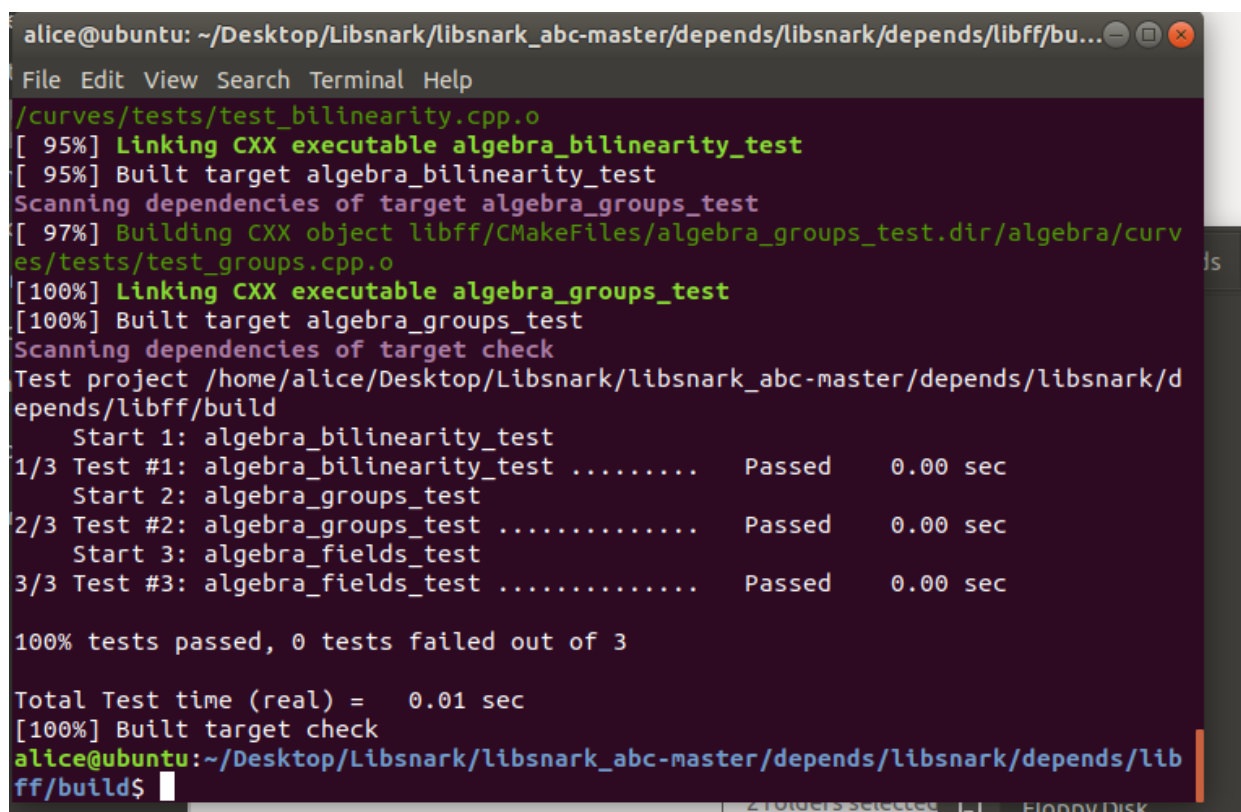
```
File Edit View Search Terminal Help
alice@ubuntu:~/Desktop/Libsnark/libsnark_abc-master/depends/libsnark/depends/lib
snark-supercop$ ./do
ar: creating ../lib/libsupercop.a
alice@ubuntu:~/Desktop/Libsnark/libsnark_abc-master/depends/libsnark/depends/lib
snark-supercop$
```

9) 安装子模块 gtest, 将文件复制到 depends/libsnark/depends/gtest。

10) 安装子模块 libff: 将下载得到的文件夹 Libff 内的文件复制到 /Libsnark/libsnark_abc-master/depends/libsnark/depends/libff。点击 libff->depends, 可以看到一个 ate-pairing 文件夹和一个 xbyak 文件夹, 这是 libff 需要的依赖项。打开这两个文件夹, 会发现它们是空的, 这时候需要将下载得到的 Ate-pairing 和 Xbyak 内的文件复制到这两个文件夹下。在 /Libsnark/libsnark_abc-master/depends/libsnark/depends/libff 下打开终端, 执行命令:

```
1 mkdir build && cd build
2 cmake ..
3 make
4 sudo make install
5 make check
```

如下图所示则安装成功:



```
alice@ubuntu: ~/Desktop/Libsnark/libsnark_abc-master/depends/libsnark/depends/libff/bu...
File Edit View Search Terminal Help
/curses/tests/test_bilinearity.cpp.o
[ 95%] Linking CXX executable algebra_bilinearity_test
[ 95%] Built target algebra_bilinearity_test
Scanning dependencies of target algebra_groups_test
[ 97%] Building CXX object libff/CMakeFiles/algebra_groups_test.dir/algebra/curv
es/tests/test_groups.cpp.o
[100%] Linking CXX executable algebra_groups_test
[100%] Built target algebra_groups_test
Scanning dependencies of target check
Test project /home/alice/Desktop/Libsnark/libsnark_abc-master/depends/libsnark/d
eends/libff/build
  Start 1: algebra_bilinearity_test
1/3 Test #1: algebra_bilinearity_test ..... Passed    0.00 sec
  Start 2: algebra_groups_test
2/3 Test #2: algebra_groups_test ..... Passed    0.00 sec
  Start 3: algebra_fields_test
3/3 Test #3: algebra_fields_test ..... Passed    0.00 sec

100% tests passed, 0 tests failed out of 3

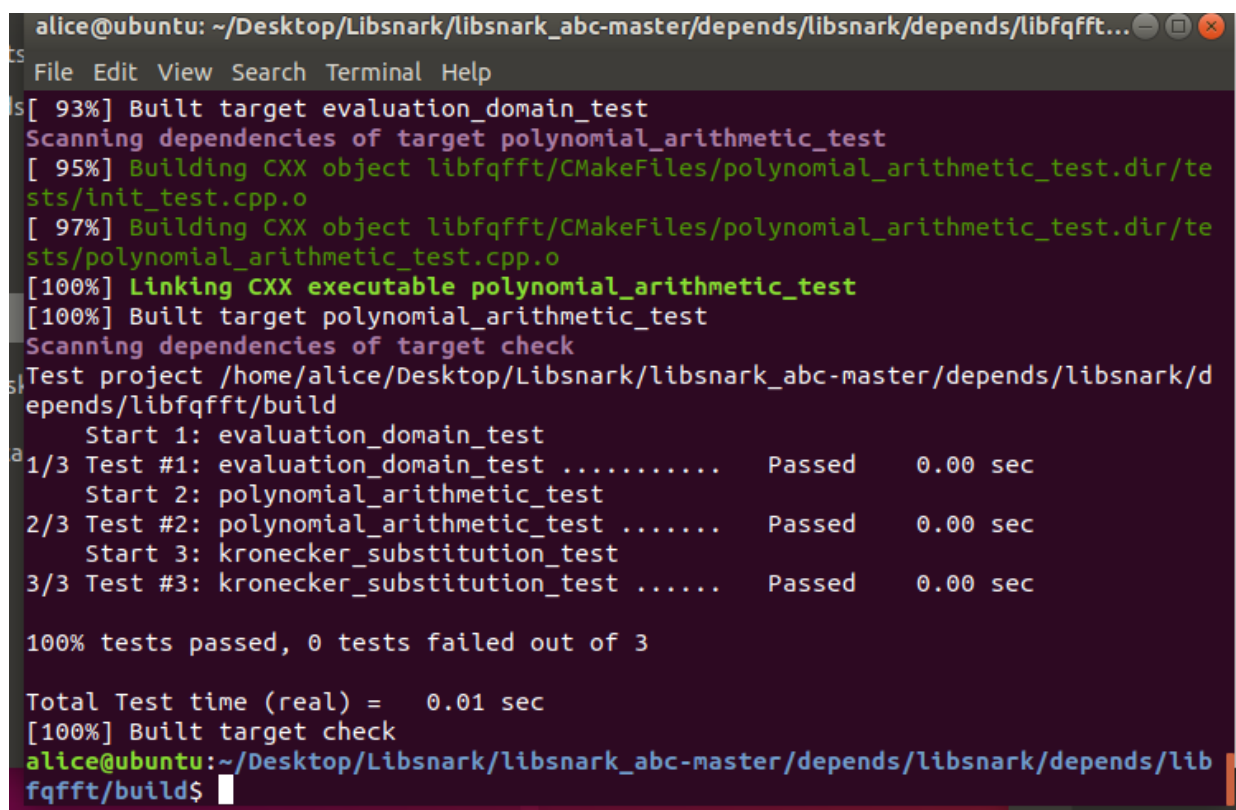
Total Test time (real) =  0.01 sec
[100%] Built target check
alice@ubuntu:~/Desktop/Libsnark/libsnark_abc-master/depends/libsnark/depends/lib
ff/build$
```

11) 安装子模块 libqfft: 将下载得到的文件夹 Libqfft 内的文件复制到 /Libsnark/libsnark_abc-master/depends/libsnark/depends/libqfft。点击 libqfft->depends, 可以看到 libqfft 有四个依赖项, 分别是 ate-pairing、gtest、libff、xbyak, 点

开来依然是空的。和上一步一样，将下载得到的文件夹内文件复制到对应文件夹下。注意 libff 里还有 depends 文件夹，里面的 ate-pairing 和 xbyaky 也是空的，需要将下载得到的 airing 和 Xbyak 文件夹内的文件复制进去。

```
1 mkdir build && cd build
2 cmake ..
3 make
4 sudo make install
5 make check
```

如下图所示则安装成功：



```
alice@ubuntu: ~/Desktop/Libsnark/libsnark_abc-master/depends/libsnark/depends/libfqfft...
File Edit View Search Terminal Help
[ 93%] Built target evaluation_domain_test
Scanning dependencies of target polynomial_arithmetic_test
[ 95%] Building CXX object libfqfft/CMakeFiles/polynomial_arithmetic_test.dir/te
sts/init_test.cpp.o
[ 97%] Building CXX object libfqfft/CMakeFiles/polynomial_arithmetic_test.dir/te
sts/polynomial_arithmetic_test.cpp.o
[100%] Linking CXX executable polynomial_arithmetic_test
[100%] Built target polynomial_arithmetic_test
Scanning dependencies of target check
Test project /home/alice/Desktop/Libsnark/libsnark_abc-master/depends/libsnark/d
eends/libfqfft/build
  Start 1: evaluation_domain_test
1/3 Test #1: evaluation_domain_test ..... Passed    0.00 sec
  Start 2: polynomial_arithmetic_test
2/3 Test #2: polynomial_arithmetic_test ..... Passed    0.00 sec
  Start 3: kronecker_substitution_test
3/3 Test #3: kronecker_substitution_test ..... Passed    0.00 sec

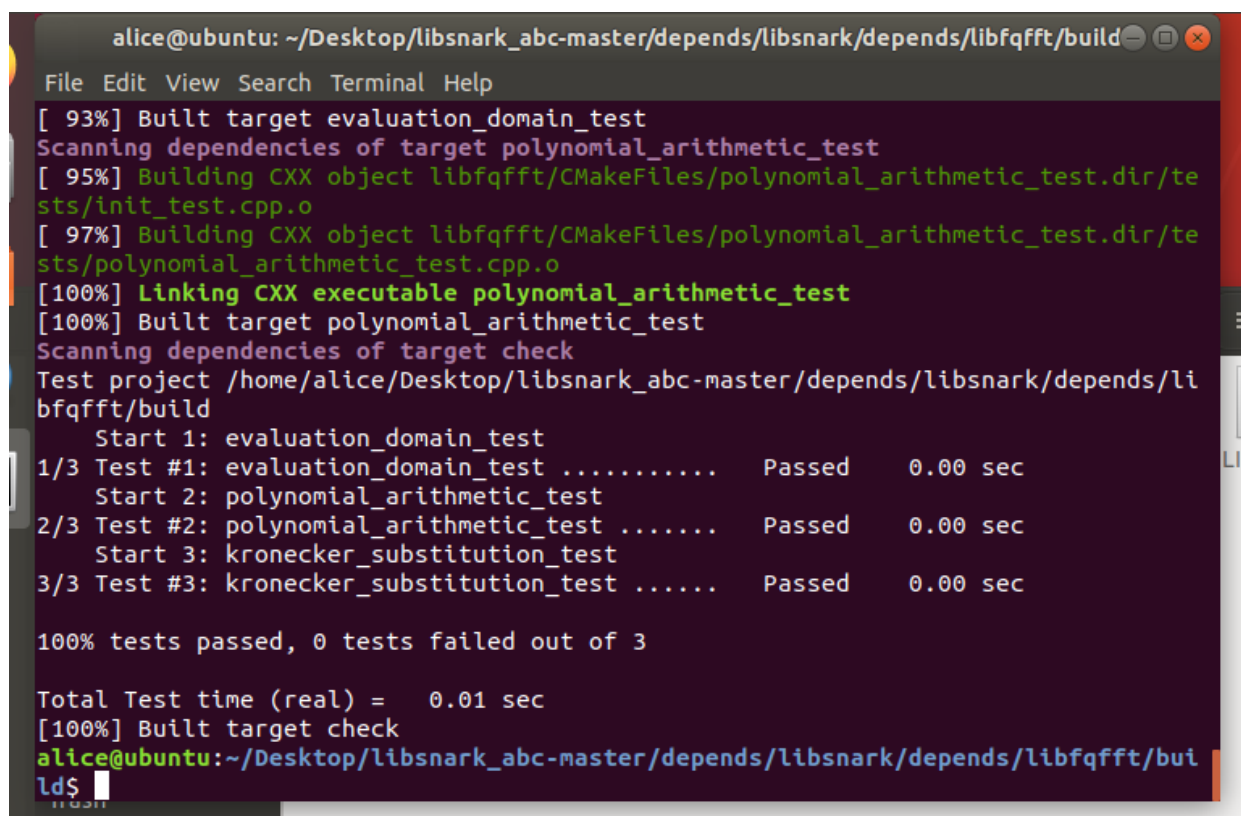
100% tests passed, 0 tests failed out of 3

Total Test time (real) =  0.01 sec
[100%] Built target check
alice@ubuntu:~/Desktop/Libsnark/libsnark_abc-master/depends/libsnark/depends/lib
fqfft/build$
```

12) libsnark 编译安装：在 /Libsnark/libsnark_abc-master/depends/libsnark 下打开终端，执行以下命令：

```
1 # 在 depends/libsnark 目录下
2 mkdir build && cd build
3 cmake ..
4 make
5 make check
```

如下图所示则安装成功：



```
alice@ubuntu: ~/Desktop/libsnark_abc-master/depends/libsnark/depends/libfqfft/build
File Edit View Search Terminal Help
[ 93%] Built target evaluation_domain_test
Scanning dependencies of target polynomial_arithmetic_test
[ 95%] Building CXX object libfqfft/CMakeFiles/polynomial_arithmetic_test.dir/te
sts/init_test.cpp.o
[ 97%] Building CXX object libfqfft/CMakeFiles/polynomial_arithmetic_test.dir/te
sts/polynomial_arithmetic_test.cpp.o
[100%] Linking CXX executable polynomial_arithmetic_test
[100%] Built target polynomial_arithmetic_test
Scanning dependencies of target check
Test project /home/alice/Desktop/libsnark_abc-master/depends/libsnark/depends/li
bfqfft/build
  Start 1: evaluation_domain_test
1/3 Test #1: evaluation_domain_test ..... Passed    0.00 sec
  Start 2: polynomial_arithmetic_test
2/3 Test #2: polynomial_arithmetic_test ..... Passed    0.00 sec
  Start 3: kronecker_substitution_test
3/3 Test #3: kronecker_substitution_test ..... Passed    0.00 sec

100% tests passed, 0 tests failed out of 3

Total Test time (real) =  0.01 sec
[100%] Built target check
alice@ubuntu:~/Desktop/libsnark_abc-master/depends/libsnark/depends/libfqfft/bui
ld$
```

13) 整体编译安装:

```
1 # 在 ~/Libsnark/libsnark_abc-master 目录下
2 mkdir build && cd build
3 cmake ..
4 make
```

14) 运行环境测试:

```
1 ./src/test
```

最终出现如下日志,则说明已顺利拥有了 zkSNARK 应用开发环境,并成功跑了第一个 zk-SNARKs 的 demo。

```

alice@ubuntu: ~/Desktop/libsnark_abc-master/build
File Edit View Search Terminal Help
    (leave) Call to alt_bn128_final_exponentiation      [0.0016s x0.93](
1774951826.5159s x0.00 from start)
    (leave) Check QAP divisibility                      [0.0042s x0.95] (1774951
826.5159s x0.00 from start)
    (leave) Online pairing computations                 [0.0042s x0.94] (1774951
826.5159s x0.00 from start)
    (leave) Call to r1cs_gg_ppzksnark_online_verifier_weak_IC [0.0043s x0.94](
1774951826.5159s x0.00 from start)
    (leave) Call to r1cs_gg_ppzksnark_online_verifier_strong_IC [0.0043s x0.93](
1774951826.5159s x0.00 from start)
    (leave) Call to r1cs_gg_ppzksnark_verifier_strong_IC [0.0051s x0.93] (1774951
826.5159s x0.00 from start)
Number of R1CS constraints: 4
Primary (public) input: 1
35

Auxiliary (private) input: 4
3
9
27
30

Verification status: 1
alice@ubuntu:~/Desktop/libsnark_abc-master/build$

```

3.2 算术电路设计与拍平

针对待证明方程 $x^3 + x + 5 = 35$ ，将其拍平成多个 $x = y$ 或者 $x = y(op)z$ 形式的等式：

- $w_1 = x \cdot x$
- $w_2 = w_1 \cdot x$
- $out = w_2 + x + 5$

3.3 电路核心代码实现 (common.hpp)

使用原型板 protoboard 搭建电路，代码放在公用文件 common.hpp 中：

Listing 1: common.hpp 核心代码

```

1 #include <libsnark/common/default_types/r1cs_gg_ppzksnark_pp.hpp>
2 #include <libsnark/zk_proof_systems/ppzksnark/r1cs_gg_ppzksnark/r1cs_gg_ppzksnark.hpp>
3 #include <libsnark/gadgetlib1/pb_variable.hpp>
4 using namespace libsnark;
5 using namespace std;
6 typedef libff::Fr<default_r1cs_gg_ppzksnark_pp> FieldT;
7
8 protoboard<FieldT> build_protoboard(int* secret) {
9     default_r1cs_gg_ppzksnark_pp::init_public_params();
10    protoboard<FieldT> pb;
11    pb_variable<FieldT> x, w_1, w_2, out;
12

```



```

13 out.allocate(pb, "out");
14 x.allocate(pb, "x");
15 w_1.allocate(pb, "w_1");
16 w_2.allocate(pb, "w_2");
17
18 pb.set_input_sizes(1);
19 pb.val(out) = 35;
20
21 // 添加 R1CS 约束满足  $a*b=c$ 
22 pb.add_r1cs_constraint(r1cs_constraint<FieldT>(x, x, w_1)); //  $x*x=w_1$ 
23 pb.add_r1cs_constraint(r1cs_constraint<FieldT>(w_1, x, w_2)); //  $w_1*x=w_2$ 
24 pb.add_r1cs_constraint(r1cs_constraint<FieldT>(w_2 + x + 5, 1, out)); //
    (w2+x+5)*1=out
25
26 if (secret != NULL) {
27     pb.val(x) = secret[0];
28     pb.val(w_1) = secret[0] * secret[0];
29     pb.val(w_2) = secret[0] * secret[0] * secret[0];
30 }
31 return pb;
32 }

```

3.4 密钥生成程序 (mysetup.cpp)

此阶段生成证明密钥 `pk.raw` 和验证密钥 `vk.raw` :

Listing 2: mysetup.cpp

```

1 #include <fstream>
2 #include "common.hpp"
3 int main() {
4     protoboard<FieldT> pb = build_protoboard(NULL);
5     const r1cs_constraint_system<FieldT> cs = pb.get_constraint_system();
6     const auto keypair =
7         r1cs_gg_ppzksnark_generator<default_r1cs_gg_ppzksnark_pp>(cs);
8     fstream pk("pk.raw", ios_base::out); pk << keypair.pk; pk.close();
9     fstream vk("vk.raw", ios_base::out); vk << keypair.vk; vk.close();
10    return 0;
11 }

```

3.5 证明生成程序 (myprove.cpp)

证明方 Alice 输入解 $x = 3$ 构造证明文件 `proof.raw` :

Listing 3: myprove.cpp

```

1 #include <fstream>
2 #include "common.hpp"
3 int main() {

```

```

4   int x; cout << "输入秘密值 x: "; cin >> x;
5   int secret[3] = {x, x*x, x*x*x};
6   protoboard<FieldT> pb = build_protoboard(secret);
7   fstream f_pk("pk.raw", ios_base::in);
8   rlcs_gg_ppzksnark_proving_key<libff::default_ec_pp> pk; f_pk >> pk; f_pk.close();
9   const auto proof = rlcs_gg_ppzksnark_prover<default_rlcs_gg_ppzksnark_pp>(pk,
    pb.primary_input(), pb.auxiliary_input());
10  fstream pr("proof.raw", ios_base::out); pr << proof; pr.close();
11  return 0;
12 }

```

3.6 验证程序 (myverify.cpp)

验证方使用 `vk.raw` 与公有输入对证明进行校验:

Listing 4: myverify.cpp

```

1  #include <fstream>
2  #include "common.hpp"
3  int main() {
4      protoboard<FieldT> pb = build_protoboard(NULL);
5      fstream f_vk("vk.raw", ios_base::in);
6      rlcs_gg_ppzksnark_verification_key<libff::default_ec_pp> vk; f_vk >> vk;
    f_vk.close();
7      fstream f_pr("proof.raw", ios_base::in);
8      rlcs_gg_ppzksnark_proof<libff::default_ec_pp> proof; f_pr >> proof; f_pr.close();
9      bool verified =
    rlcs_gg_ppzksnark_verifier_strong_IC<default_rlcs_gg_ppzksnark_pp>(vk,
    pb.primary_input(), proof);
10     cout << "验证结果: " << verified << endl;
11     return 0;
12 }

```

3.7 编译运行与配置

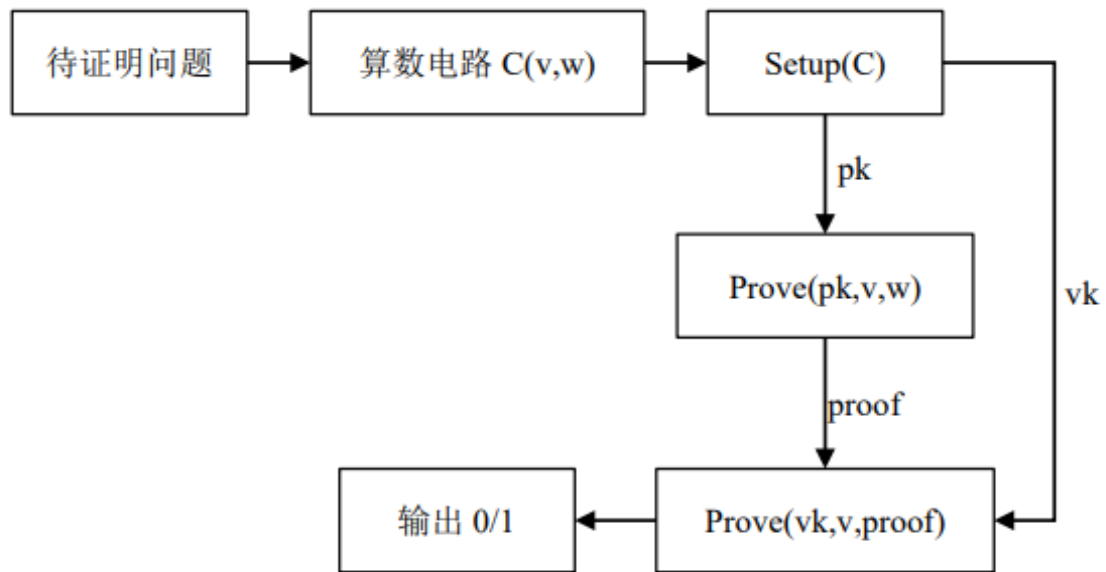
在 `src` 目录下创建上述文件。随后打开 `CMakeLists.txt`, 在末尾追加如下代码:

Listing 5: CMakeLists.txt 追加配置

```

1  # mysetup
2  add_executable(mysetup mysetup.cpp)
3  target_link_libraries(mysetup snark)
4  target_include_directories(mysetup PUBLIC ${DEPENDS_DIR}/libsnark
    ${DEPENDS_DIR}/libsnark/depends/libffqfft)
5
6  # myprove
7  add_executable(myprove myprove.cpp)
8  target_link_libraries(myprove snark)

```



```

9 target_include_directories(myprove PUBLIC ${DEPENDS_DIR}/libsna
  ${DEPENDS_DIR}/libsna/depends/libqfft)
10
11 # myverify
12 add_executable(myverify myverify.cpp)
13 target_link_libraries(myverify snark)
14 target_include_directories(myverify PUBLIC ${DEPENDS_DIR}/libsna
  ${DEPENDS_DIR}/libsna/depends/libqfft)

```

在 `build` 目录下执行编译并运行流程：

```

1 cmake ..
2 make
3 cd src
4 ./mysetup
5 ./myprove
6 # 输入 3
7 ./myverify

```

运行结果如下：验证结果为 1，表示证明方提供的私密输入 $x = 3$ 满足方程 $x^3 + x + 5 = 35$ 。该结果证实了证明方确实掌握该方程的有效解，且在不泄露 x 具体数值的情况下，通过了零知识证明的验证

```

alice@ubuntu: ~/Desktop/libsnark_abc-master/build/src
File Edit View Search Terminal Help
      (enter) Call to alt_bn128_exp_by_neg_z      [          ](
1774952407.3652s x0.00 from start)
      (leave) Call to alt_bn128_exp_by_neg_z      [0.0005s x1.00](
1774952407.3657s x0.00 from start)
      (enter) Call to alt_bn128_exp_by_neg_z      [          ](
1774952407.3657s x0.00 from start)
      (leave) Call to alt_bn128_exp_by_neg_z      [0.0005s x1.00](
1774952407.3662s x0.00 from start)
      (leave) Call to alt_bn128_final_exponentiation_last_chunk [0.0017s
x1.00] (1774952407.3662s x0.00 from start)
      (leave) Call to alt_bn128_final_exponentiation [0.0018s x1.00](
1774952407.3662s x0.00 from start)
      (leave) Check QAP divisibility [0.0043s x1.00] (1774952
407.3662s x0.00 from start)
      (leave) Online pairing computations [0.0043s x1.00] (1774952
407.3662s x0.00 from start)
      (leave) Call to r1cs_gg_ppzksnark_online_verifier_weak_IC [0.0044s x1.00](
1774952407.3663s x0.00 from start)
      (leave) Call to r1cs_gg_ppzksnark_online_verifier_strong_IC [0.0044s x1.00](
1774952407.3663s x0.00 from start)
      (leave) Call to r1cs_gg_ppzksnark_verifier_strong_IC [0.0051s x1.00] (1774952
407.3663s x0.00 from start)
验证结果: 1
alice@ubuntu:~/Desktop/libsnark_abc-master/build/src$

```

4 总结

通过复刻教材步骤，我完整实现了针对方程 $x^3 + x + 5 = 35$ 的零知识证明系统。实验加深了对电路拍平、R1CS 约束构建以及 zkSNARK 三阶段流程的理解。虽然环境配置过程复杂，但最终通过 Setup-Prove-Verify 全流程，成功验证了证明方的解，充分体现了零知识证明在数据安全领域的应用价值。